UNIVERSIDAD CATÓLICA DE SANTA MARÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

LENGUAJES DE PROGRAMACIÓN III

PRÁCTICA N° 06: MVC EN JAVA

Código	Apellidos y Nombres	Fecha
2025002762	Rodríguez Fádel, Gian Piero Khalil	08/10/2026

1. ACTIVIDADES

Las actividades 1, 2 y 3 se desarrollan de forma acumulativa en el proyecto de pedidos. Elegí los ejercicios 1 y 2 porque el segundo amplía directamente el carrito del primero. El código conserva los paquetes modelo, vista y controlador, los nombres en español y los recorridos directos del repositorio original.

1.1 Actividad 1 Registro y listado de pedidos

La aplicación conserva el caso del restaurante planteado en la guía. Main crea el modelo, la vista y el controlador. El menú permite solicitar el nombre y el tipo de plato, registrar el pedido y mostrar los pedidos actuales. ArrayList permite añadir pedidos sin fijar de antemano la cantidad de registros.

La separación de datos, presentación e interacción corresponde al propósito de MVC descrito por Syromiatnikov y Weyns (2014). En esta implementación de consola, el controlador solicita las entradas a la vista, llama al modelo y entrega el resultado nuevamente a la vista.

1.2 Actividad 2 Operaciones adicionales

Se incorporaron la búsqueda por nombre y por tipo, la actualización del nombre de un pedido pendiente, la eliminación y el conteo total y por tipo. La búsqueda compara cadenas completas con equalsIgnoreCase y elimina espacios externos con trim. Una búsqueda sin coincidencias devuelve una lista vacía; la vista informa el resultado sin recibir null.

Cada pedido tiene un ID que se asigna una sola vez. La selección del usuario se hace mediante ese ID y no mediante una posición del ArrayList. Por eso eliminar el pedido 1 no convierte al pedido 2 en otro registro. TreeMap agrupa los tipos sin distinguir mayúsculas y minúsculas y mantiene un orden alfabético al presentar los conteos.

1.3 Actividad 3 Estados e historial

Todo pedido comienza como PENDIENTE. La opción de completar cambia su estado a COMPLETO y la eliminación lo cambia a ELIMINADO. La lista actual incluye pendientes y completos, mientras que los eliminados se excluyen de búsquedas y conteos actuales. El filtro por estado y el contador de pendientes consultan el mismo conjunto de datos.

Al completar o eliminar se añade al historial una copia del pedido con el estado de ese momento. Si un pedido completo se elimina después, el historial conserva las dos transiciones. Se impide completar dos veces un pedido y modificar el nombre de un pedido servido. Estas reglas se aplican en el modelo.

Correcciones del código inicial

Problema encontrado	**Corrección**
Buscar por tipo comparaba el nombre del plato.	Se compara getTipo con el tipo solicitado.
Un índice inválido seguía llegando a remove.	El ID se valida y se rechaza antes de modificar datos.
La opción de buscar no tenía break.	Cada operación termina su caso sin ejecutar el siguiente.
El do while terminaba con la opción de eliminar.	El programa termina exclusivamente con 0 o al finalizar la entrada.
Faltaban actualización, estados, historial y Main.	Se implementaron las funciones y la clase de inicio.
El modelo imprimía errores y podía devolver null.	El modelo devuelve datos o una excepción; la vista muestra los mensajes.

Explicación de los archivos de la actividad

Archivo: lab6_actividad1/src/modelo/Pedido.java

```
package modelo;

public class Pedido {
  private int id;
  private String nombrePlato;
  private String tipo;
  private EstadoPedido estado;

  public Pedido(int id, String nombrePlato, String tipo)
  {
      this.id = id;
      this.nombrePlato = nombrePlato.trim();
      this.tipo = tipo.trim();
      estado = EstadoPedido.PENDIENTE;
  }

  public int getId() {
      return id;
  }
```

```
    public String getNombrePlato() {
        return nombrePlato;
    }

    public void setNombrePlato(String nombrePlato) {
        this.nombrePlato = nombrePlato.trim();
    }

    public String getTipo() {
        return tipo;
    }

    public EstadoPedido getEstado() {
        return estado;
    }

    public void setEstado(EstadoPedido estado) {
        this.estado = estado;
    }
}
```

Explicación del archivo: Pedido.java

Representa un pedido individual. Contiene id, nombrePlato, tipo y estado. El constructor recibe el identificador, el nombre y el tipo; limpia los espacios externos y establece PENDIENTE como estado inicial. Los getters permiten consultar el pedido. setNombrePlato cambia el nombre y setEstado registra una transición.

El ID no tiene setter, de modo que permanece estable. El tipo se mantiene después del registro porque la actividad solicita actualizar el nombre. Esta clase no imprime mensajes ni lee el teclado. Las reglas para modificar o completar un pedido se comprueban en PedidoModelo antes de llamar a sus setters.

Archivo: lab6_actividad1/src/modelo/EstadoPedido.java

```
package modelo;

public enum EstadoPedido {
  PENDIENTE, COMPLETO, ELIMINADO
}
```

Explicación del archivo: EstadoPedido.java

Define los tres valores permitidos: PENDIENTE, COMPLETO y ELIMINADO. Usé un enum para evitar escribir el estado como una cadena que pudiera contener errores de ortografía o combinaciones no previstas. Pedido guarda un valor de esta enumeración y PedidoModelo lo utiliza al filtrar registros, validar cambios y contar pendientes.

Archivo: lab6_actividad1/src/modelo/PedidoModelo.java

```
package modelo;

import java.util.ArrayList;
import java.util.List;
import java.util.TreeMap;

public class PedidoModelo {
  private List<Pedido> pedidos;
  private List<Pedido> historial;
  private int siguienteId;

  public PedidoModelo()
  {
      pedidos = new ArrayList<>();
      historial = new ArrayList<>();
      siguienteId = 1;
  }

  public void agregarPedido(String nombre, String tipo)
  {
      if(nombre.trim().isEmpty() || tipo.trim().isEmpty())
      {
          throw new IllegalArgumentException("El nombre y el tipo no pueden estar vacíos.");
      }
      pedidos.add(new Pedido(siguienteId, nombre, tipo));
      siguienteId++;
  }

  public Pedido buscarPorId(int id)
  {
      for(Pedido pedido: pedidos)
      {
          if(pedido.getId() == id && pedido.getEstado() != EstadoPedido.ELIMINADO)
          {
              return pedido;
          }
      }
      throw new IllegalArgumentException("Número de pedido inválido.");
  }

  public void eliminarPedido(int id)
  {
      Pedido pedido = buscarPorId(id);
      pedido.setEstado(EstadoPedido.ELIMINADO);
      registrarHistorial(pedido);
  }

  private void registrarHistorial(Pedido pedido)
  {
```

```
        Pedido registro = new Pedido(pedido.getId(), pedido.getNombrePlato(), pedido.getTipo());
        registro.setEstado(pedido.getEstado());
        historial.add(registro);
    }

    public void actualizarPedido(int id, String nombre)
    {
        Pedido pedido = buscarPorId(id);
        if(pedido.getEstado() != EstadoPedido.PENDIENTE)
        {
            throw new IllegalArgumentException("Solo se pueden actualizar pedidos pendientes.");
        }
        if(nombre.trim().isEmpty())
        {
            throw new IllegalArgumentException("El nombre no puede estar vacío.");
        }
        pedido.setNombrePlato(nombre);
    }

    public List<Pedido> buscarPedido(String nombre)
    {
        List<Pedido> pedidosBuscados = new ArrayList<>();
        for(Pedido pedido: getPedidos())
        {
            if(pedido.getNombrePlato().equalsIgnoreCase(nombre.trim()))
            {
                pedidosBuscados.add(pedido);
            }
        }
        return pedidosBuscados;
    }

    public List<Pedido> buscarPedidoTipo(String tipo)
    {
        List<Pedido> pedidosBuscados = new ArrayList<>();
        for(Pedido pedido: getPedidos())
        {
            if(pedido.getTipo().equalsIgnoreCase(tipo.trim()))
            {
                pedidosBuscados.add(pedido);
            }
        }
        return pedidosBuscados;
    }

    public TreeMap<String, Integer> contarPedidos()
    {
        TreeMap<String, Integer> pedidosContados = new
TreeMap<>(String.CASE_INSENSITIVE_ORDER);
        for(Pedido pedido: getPedidos())
        {
```

```
            String tipo = pedido.getTipo();
            if(pedidosContados.containsKey(tipo))
            {
                pedidosContados.put(tipo, pedidosContados.get(tipo) + 1);
            }
            else
            {
                pedidosContados.put(tipo, 1);
            }
        }
        return pedidosContados;
    }

    public void completarPedido(int id)
    {
        Pedido pedido = buscarPorId(id);
        if(pedido.getEstado() != EstadoPedido.PENDIENTE)
        {
            throw new IllegalArgumentException("El pedido ya está completo.");
        }
        pedido.setEstado(EstadoPedido.COMPLETO);
        registrarHistorial(pedido);
    }

    public List<Pedido> pedidosPorEstado(EstadoPedido estado)
    {
        List<Pedido> resultado = new ArrayList<>();
        for(Pedido pedido: pedidos)
        {
            if(pedido.getEstado() == estado)
            {
                resultado.add(pedido);
            }
        }
        return resultado;
    }

    public int contarPendientes()
    {
        return pedidosPorEstado(EstadoPedido.PENDIENTE).size();
    }

    public List<Pedido> getHistorial()
    {
        return new ArrayList<>(historial);
    }

    public List<Pedido> getPedidos()
    {
        List<Pedido> actuales = new ArrayList<>();
        for(Pedido pedido: pedidos)
```

```
        {
            if(pedido.getEstado() != EstadoPedido.ELIMINADO)
            {
                actuales.add(pedido);
            }
        }
        return actuales;
    }
}
```

Explicación del archivo: PedidoModelo.java

Administra dos listas: pedidos conserva los registros y su estado actual; historial conserva copias de las transiciones. siguienteId comienza en 1 y aumenta después de un registro válido. agregarPedido valida nombre y tipo. buscarPorId recorre la lista y rechaza IDs inexistentes o eliminados. eliminarPedido cambia el estado y registrarHistorial crea la copia que conserva el resultado.

actualizarPedido comprueba que el pedido siga pendiente y que el nuevo nombre tenga contenido. buscarPedido y buscarPedidoTipo recorren getPedidos y comparan el campo correspondiente. contarPedidos construye un TreeMap con un contador por tipo; cuando ya existe una clave aumenta su valor y, en caso contrario, la inicia en 1.

completarPedido valida el estado anterior, cambia a COMPLETO y registra el evento. pedidosPorEstado devuelve una lista filtrada; contarPendientes usa su tamaño. getPedidos construye la lista de registros no eliminados y getHistorial devuelve otra lista con las copias guardadas. El modelo no depende de la vista ni del controlador, y no contiene System.out.

Archivo: lab6_actividad1/src/vista/PedidoVista.java

```
package vista;

import java.util.List;
import java.util.Map;
import java.util.Scanner;
import modelo.Pedido;

public class PedidoVista {
  private Scanner scanner;

  public PedidoVista()
  {
      scanner = new Scanner(System.in);
  }

  public String solicitarTexto(String mensaje)
  {
      System.out.print(mensaje);
      if(!scanner.hasNextLine())
      {
          return null;
```

```
    }
    return scanner.nextLine();
}

public String solicitarNombrePlato()
{
    return solicitarTexto("Introduce el nombre del plato: ");
}

public String solicitarTipoPlato()
{
    return solicitarTexto("Introduce el tipo del plato: ");
}

public int solicitarNumero(String mensaje)
{
    String dato = solicitarTexto(mensaje);
    if(dato == null)
    {
        throw new IllegalArgumentException("No se recibió un número.");
    }
    try
    {
        return Integer.parseInt(dato.trim());
    }
    catch(NumberFormatException e)
    {
        throw new IllegalArgumentException("Debes ingresar un número entero.");
    }
}

public void mostrarPedidos(List<Pedido> pedidos)
{
    if(pedidos.isEmpty())
    {
        mostrarMensaje("No hay pedidos en la lista.");
        return;
    }
    mostrarMensaje("ID | Plato | Tipo | Estado");
    for(Pedido pedido: pedidos)
    {
        mostrarMensaje(pedido.getId() + " | " + pedido.getNombrePlato() + " | "
                + pedido.getTipo() + " | " + pedido.getEstado());
    }
}

public void mostrarConteo(int total, Map<String, Integer> conteo)
{
    mostrarMensaje("Total de pedidos actuales: " + total);
    for(Map.Entry<String, Integer> entrada: conteo.entrySet())
    {
```

```
            mostrarMensaje(entrada.getKey() + ": " + entrada.getValue());
        }
    }

    public void mostrarMenu()
    {
        mostrarMensaje("\nGESTIÓN DE PEDIDOS");
        mostrarMensaje("1. Agregar pedido");
        mostrarMensaje("2. Mostrar pedidos");
        mostrarMensaje("3. Eliminar pedido");
        mostrarMensaje("4. Buscar pedido");
        mostrarMensaje("5. Actualizar nombre");
        mostrarMensaje("6. Contar pedidos");
        mostrarMensaje("7. Marcar como completo");
        mostrarMensaje("8. Mostrar pedidos por estado");
        mostrarMensaje("9. Contar pendientes");
        mostrarMensaje("10. Historial");
        mostrarMensaje("0. Salir");
    }

    public void mostrarMenuBuscar()
    {
        mostrarMensaje("1. Nombre");
        mostrarMensaje("2. Tipo");
    }

    public String solicitarOpcion()
    {
        return solicitarTexto("Opción: ");
    }

    public void mostrarMensaje(String mensaje)
    {
        System.out.println(mensaje);
    }

    public void cerrarScanner()
    {
        scanner.close();
    }
}
```

Explicación del archivo: PedidoVista.java

Se encarga de la consola. Scanner lee líneas completas, lo que evita mezclar nextInt con nextLine. solicitarTexto presenta un mensaje y devuelve la entrada, o null cuando termina el flujo. solicitarNumero convierte una línea con Integer.parseInt y transforma un error de formato en un mensaje comprensible.

mostrarPedidos presenta ID, plato, tipo y estado, o indica que la lista está vacía. mostrarConteo recibe el total y el mapa por tipo, sin calcular reglas de negocio. mostrarMenu y

mostrarMenuBuscar enumeran las opciones. solicitarOpcion lee la selección y mostrarMensaje imprime las respuestas del controlador. cerrarScanner libera el lector al salir. Los métodos solicitarNombrePlato y solicitarTipoPlato conservan las entradas específicas del ejemplo de la guía.

Archivo: lab6_actividad1/src/controlador/PedidoControlador.java

```
package controlador;

import modelo.EstadoPedido;
import modelo.PedidoModelo;
import vista.PedidoVista;

public class PedidoControlador {
  private PedidoModelo modelo;
  private PedidoVista vista;

  public PedidoControlador(PedidoModelo modelo, PedidoVista vista)
  {
      this.modelo = modelo;
      this.vista = vista;
  }

  private String solicitarTexto(String mensaje)
  {
      String dato = vista.solicitarTexto(mensaje);
      if(dato == null)
      {
          throw new IllegalArgumentException("Entrada incompleta.");
      }
      return dato;
  }

  public void buscarPedidos()
  {
      vista.mostrarMenuBuscar();
      String opcion = vista.solicitarOpcion();
      if("1".equals(opcion))
      {
          vista.mostrarPedidos(modelo.buscarPedido(solicitarTexto("Nombre: ")));
      }
      else if("2".equals(opcion))
      {
          vista.mostrarPedidos(modelo.buscarPedidoTipo(solicitarTexto("Tipo: ")));
      }
      else
      {
          vista.mostrarMensaje("Opción de búsqueda inválida.");
      }
  }

  public void mostrarPorEstado()
```

```
{
    vista.mostrarMensaje("1. Pendientes");
    vista.mostrarMensaje("2. Completos");
    String opcion = vista.solicitarOpcion();
    if("1".equals(opcion))
    {
        vista.mostrarPedidos(modelo.pedidosPorEstado(EstadoPedido.PENDIENTE));
    }
    else if("2".equals(opcion))
    {
        vista.mostrarPedidos(modelo.pedidosPorEstado(EstadoPedido.COMPLETO));
    }
    else
    {
        vista.mostrarMensaje("Estado inválido.");
    }
}

public void iniciar()
{
    String opcion;
    do
    {
        vista.mostrarMenu();
        opcion = vista.solicitarOpcion();
        if(opcion == null)
        {
            break;
        }
        try
        {
            switch(opcion)
            {
            case "1":
                String nombre = solicitarTexto("Introduce el nombre del plato: ");
                String tipo = solicitarTexto("Introduce el tipo del plato: ");
                modelo.agregarPedido(nombre, tipo);
                vista.mostrarMensaje("Pedido agregado: " + nombre.trim());
                break;
            case "2":
                vista.mostrarPedidos(modelo.getPedidos());
                break;
            case "3":
                vista.mostrarPedidos(modelo.getPedidos());
                modelo.eliminarPedido(vista.solicitarNumero("ID a eliminar: "));
                vista.mostrarMensaje("Pedido eliminado.");
                break;
            case "4":
                buscarPedidos();
                break;
            case "5":
```

```
                int id = vista.solicitarNumero("ID a actualizar: ");
                modelo.actualizarPedido(id, solicitarTexto("Nuevo nombre: "));
                vista.mostrarMensaje("Pedido actualizado.");
                break;
            case "6":
                vista.mostrarConteo(modelo.getPedidos().size(), modelo.contarPedidos());
                break;
            case "7":
                modelo.completarPedido(vista.solicitarNumero("ID a completar: "));
                vista.mostrarMensaje("Pedido completo.");
                break;
            case "8":
                mostrarPorEstado();
                break;
            case "9":
                vista.mostrarMensaje("Pedidos pendientes: " + modelo.contarPendientes());
                break;
            case "10":
                vista.mostrarPedidos(modelo.getHistorial());
                break;
            case "0":
                vista.mostrarMensaje("Saliendo...");
                break;
            default:
                vista.mostrarMensaje("Opción no válida.");
            }
        }
        catch(IllegalArgumentException e)
        {
            vista.mostrarMensaje(e.getMessage());
        }
    } while(!opcion.equals("0"));
    vista.cerrarScanner();
  }
}
```

Explicación del archivo: PedidoControlador.java

Recibe PedidoModelo y PedidoVista mediante el constructor. iniciar mantiene un do while con un switch, solicita los datos de cada operación, llama al modelo y presenta el resultado. El método auxiliar solicitarTexto detecta entradas incompletas; las excepciones de validación se capturan para que el menú continúe.

buscarPedidos selecciona nombre o tipo y pasa a la vista la lista que devuelve el modelo. mostrarPorEstado permite escoger pendientes o completos. Los casos de registrar, eliminar, actualizar, contar y completar llaman a los métodos del modelo; los casos de pendientes e historial muestran sus resultados. Cada caso tiene break y 0 termina el ciclo. Al finalizar se cierra el Scanner de la vista.

Archivo: lab6_actividad1/src/principal/Main.java

```
package principal;

import modelo.PedidoModelo;
import vista.PedidoVista;
import controlador.PedidoControlador;

public class Main {
  public static void main(String[] args)
  {
      PedidoModelo modelo = new PedidoModelo();
      PedidoVista vista = new PedidoVista();
      PedidoControlador controlador = new PedidoControlador(modelo, vista);
      controlador.iniciar();
  }
}
```

Explicación del archivo: Main.java

Contiene el punto de entrada main. Instancia PedidoModelo, después PedidoVista y finalmente PedidoControlador con esas dos referencias. Luego llama a iniciar. Separar el arranque permite localizar rápidamente qué objetos se necesitan y evita colocar el menú o las reglas de pedidos en la clase principal.

Archivo: lab6_actividad1/src/module-info.java

```
module laboratorio.pedidos {
}
```

Explicación del archivo: module-info.java

Declara el módulo laboratorio.pedidos. No necesita requires adicionales porque las colecciones y Scanner pertenecen a java.base, disponible de forma implícita. El nombre del módulo utiliza palabras sin dígitos finales para evitar advertencias del compilador. La ejecución por classpath indicada en el anexo también funciona.

Archivos de configuración del proyecto de pedidos

Archivo: lab6_actividad1/.project

```
<?xml version="1.0" encoding="UTF-8"?>
<projectDescription>
  <name>lab6_actividad1</name>
  <comment></comment>
  <projects>
  </projects>
  <buildSpec>
      <buildCommand>
          <name>org.eclipse.jdt.core.javabuilder</name>
          <arguments>
          </arguments>
      </buildCommand>
```

```
  </buildSpec>
  <natures>
      <nature>org.eclipse.jdt.core.javanature</nature>
  </natures>
</projectDescription>
```

Explicación del archivo: .project

Identifica el proyecto de Eclipse mediante su nombre. buildSpec utiliza org.eclipse.jdt.core.javabuilder para compilar Java y natures declara org.eclipse.jdt.core.javanature para que el IDE lo reconozca como proyecto Java. Se conserva una configuración simple sin dependencias ni constructores adicionales.

Archivo: lab6_actividad1/.classpath

```
<?xml version="1.0" encoding="UTF-8"?>
<classpath>
  <classpathentry kind="con"
path="org.eclipse.jdt.launching.JRE_CONTAINER/org.eclipse.jdt.internal.debug.ui.launcher.Sta
ndardVMType/JavaSE-17">
      <attributes>
          <attribute name="module" value="true"/>
      </attributes>
  </classpathentry>
  <classpathentry kind="src" path="src"/>
  <classpathentry kind="output" path="bin"/>
</classpath>
```

Explicación del archivo: .classpath

Declara src como carpeta de entrada, el contenedor de JavaSE-17 como biblioteca del sistema y bin como carpeta de salida. Eclipse puede compilar el proyecto a partir de estas rutas. Cada proyecto mantiene su propia salida para evitar mezclar las dos clases principal.Main.

Archivo: lab6_actividad1/.settings/org.eclipse.core.resources.prefs

```
eclipse.preferences.version=1
encoding/<project>=UTF-8
```

Explicación del archivo: org.eclipse.core.resources.prefs

Establece UTF-8 como codificación del proyecto. Es necesaria para conservar las tildes, la ñ y los mensajes de consola en español. eclipse.preferences.version identifica la versión del archivo de preferencias.

Archivo: lab6_actividad1/.settings/org.eclipse.jdt.core.prefs

```
eclipse.preferences.version=1
org.eclipse.jdt.core.compiler.codegen.targetPlatform=17
org.eclipse.jdt.core.compiler.codegen.unusedLocal=preserve
org.eclipse.jdt.core.compiler.compliance=17
```

```
org.eclipse.jdt.core.compiler.debug.lineNumber=generate
org.eclipse.jdt.core.compiler.debug.localVariable=generate
org.eclipse.jdt.core.compiler.debug.sourceFile=generate
org.eclipse.jdt.core.compiler.problem.enablePreviewFeatures=disabled
org.eclipse.jdt.core.compiler.problem.reportPreviewFeatures=warning
org.eclipse.jdt.core.compiler.release=enabled
org.eclipse.jdt.core.compiler.source=17
```

Explicación del archivo: org.eclipse.jdt.core.prefs

Configura Java 17 como nivel de fuente, compatibilidad y destino de las clases. Desactiva las características preview para que el código pueda ejecutarse con una instalación normal de JDK 17. Se usa el mismo nivel en los dos proyectos y en la compilación de verificación.

Archivo: lab6_actividad1/fuentes.txt

```
src/controlador/PedidoControlador.java
src/modelo/EstadoPedido.java
src/modelo/Pedido.java
src/modelo/PedidoModelo.java
src/module-info.java
src/principal/Main.java
src/vista/PedidoVista.java
```

Explicación del archivo: fuentes.txt

Enumera todos los archivos Java del proyecto mediante rutas relativas. El compilador acepta @fuentes.txt y lee cada ruta como un argumento, de modo que el comando de compilación incluye modelos, vistas, controladores, Main y module-info sin depender de un comodín específico del sistema operativo.

Opciones de la actividad

Opción	**Resultado**
1 y 2	Registrar un pedido y mostrar los pedidos actuales.
3 y 4	Eliminar por ID y buscar por nombre o tipo.
5 y 6	Actualizar el nombre y contar total y por tipo.
7 y 8	Completar un pedido y mostrar pendientes o completos.
9 y 10	Contar pendientes y mostrar el historial.
0	Finalizar el programa.

Compilación y ejecución de las actividades

Las figuras muestran la salida registrada al compilar y ejecutar los archivos de este informe. Las imágenes de resultados omiten la repetición del menú para facilitar la lectura; las transcripciones completas se conservan en pruebas/evidencias.

Compilación del proyecto de pedidos

El compilador de Java genera las clases de modelos, vista, controlador y Main. La opción -verbose permite comprobar los archivos .class escritos durante esta compilación real.

```
+ cd Lab6/lab6_actividad1
+ java -version
openjdk version "17.0.20" 2026-07-21
OpenJDK Runtime Environment (build 17.0.20+8-1-24.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 17.0.20+8-1-24.04-Ubuntu, mixed mode, sharing)
+ java com.sun.tools.javac.Main -verbose -encoding UTF-8 --release 17 -d
../../qa/build/actividad @fuentes.txt
[wrote ../../qa/build/actividad/module-info.class]
[wrote ../../qa/build/actividad/controlador/PedidoControlador.class]
[wrote ../../qa/build/actividad/modelo/PedidoModelo.class]
[wrote ../../qa/build/actividad/vista/PedidoVista.class]
[wrote ../../qa/build/actividad/modelo/Pedido.class]
[wrote ../../qa/build/actividad/modelo/EstadoPedido.class]
[wrote ../../qa/build/actividad/principal/Main.class]
[total 313ms]
```

Figura 1. Compilación del proyecto de pedidos. Salida registrada en una ejecución real con Java 17.

Registro y listado

Se registran Sopa como Entrada y Pollo como Fondo. El listado permite comprobar que cada registro conserva su ID y comienza pendiente.

```
Opción: 1
Introduce el nombre del plato: Sopa
Introduce el tipo del plato: Entrada
Pedido agregado: Sopa

Opción: 1
Introduce el nombre del plato: Pollo
Introduce el tipo del plato: Fondo
Pedido agregado: Pollo

Opción: 2
ID | Plato | Tipo | Estado
1 | Sopa | Entrada | PENDIENTE
2 | Pollo | Fondo | PENDIENTE
```

Figura 2. Registro y listado. Salida registrada en una ejecución real con Java 17.

Actualización búsqueda y conteo

Se cambia el nombre del pedido 2 a Pollo al horno. La búsqueda por fondo encuentra su tipo aunque se escriba en minúsculas. El conteo indica dos pedidos actuales y uno de cada tipo.

```
Opción: 5
ID a actualizar: 2
Nuevo nombre: Pollo al horno
Pedido actualizado.

Opción: 4
1. Nombre
2. Tipo
Opción: 2
Tipo: fondo
ID | Plato | Tipo | Estado
2 | Pollo al horno | Fondo | PENDIENTE

Opción: 6
Total de pedidos actuales: 2
Entrada: 1
Fondo: 1
```

Figura 3. Actualización búsqueda y conteo. Salida registrada en una ejecución real con Java 17.

Estados pendientes e historial

El pedido 1 se completa y el pedido 2 se elimina. El filtro muestra la sopa servida, el contador indica un pendiente antes de eliminar el pollo, y el historial conserva el resultado de ambas operaciones.

```
Opción: 7
ID a completar: 1
Pedido completo.

Opción: 8
1. Pendientes
2. Completos
Opción: 2
ID | Plato | Tipo | Estado
1 | Sopa | Entrada | COMPLETO

Opción: 9
Pedidos pendientes: 1

Opción: 3
ID | Plato | Tipo | Estado
1 | Sopa | Entrada | COMPLETO
2 | Pollo al horno | Fondo | PENDIENTE
ID a eliminar: 2
Pedido eliminado.

Opción: 10
ID | Plato | Tipo | Estado
1 | Sopa | Entrada | COMPLETO
2 | Pollo al horno | Fondo | ELIMINADO
```

Figura 4. Estados pendientes e historial. Salida registrada en una ejecución real con Java 17.

Validación de entradas

Un ID inexistente y una entrada que no es un número generan mensajes claros. El programa sigue funcionando y no modifica los datos debido al error.

```
Opción: 3
ID | Plato | Tipo | Estado
1 | Sopa | Entrada | COMPLETO
ID a eliminar: 99
Número de pedido inválido.

Opción: 7
ID a completar: texto
Debes ingresar un número entero.
```

Figura 5. Validación de entradas. Salida registrada en una ejecución real con Java 17.

2. EJERCICIOS

2.1 Ejercicio 1 Carrito de compras

El sistema permite registrar productos, listarlos, agregar cantidades al carrito, revisar su contenido, retirar un producto, aplicar un descuento porcentual, elegir una modalidad de envío, confirmar la compra y consultar el historial. Al agregar otra vez el mismo producto se acumulan sus unidades en una sola línea.

El dinero se maneja con BigDecimal. El precio admite hasta dos decimales; se rechazan precios no positivos o superiores a S/ 1 000 000.00. La cantidad válida está entre 1 y 1000 unidades por producto. Estas restricciones evitan valores de entrada sin sentido para la práctica.

Cálculo	**Regla utilizada**
Subtotal	Suma de precio por cantidad de cada producto.
Descuento	Subtotal por porcentaje dividido entre 100; redondeo HALF_UP a dos decimales.
Envío	RECOJO S/ 0.00, LOCAL S/ 10.00, NACIONAL S/ 25.00.
Total	Subtotal menos descuento más envío.
Compra	Requiere sesión, carrito con productos y confirmación S.

La confirmación N conserva el carrito. La confirmación S crea una Compra, copia sus líneas y guarda subtotal, descuento y envío de esa operación. Luego se vacía el carrito y se restablecen descuento y envío. Por eso una compra anterior no cambia cuando se arma otro carrito.

2.2 Ejercicio 2 Autenticación reseñas y calificaciones

El registro solicita nombre, correo y contraseña. El correo se normaliza a minúsculas y no puede repetirse. Se comprueba un formato básico de correo y una contraseña de al menos ocho caracteres. Iniciar sesión exige que el correo y la contraseña coincidan con un usuario registrado.

No se guarda la contraseña original dentro de Usuario. La clase genera una sal aleatoria de 16 bytes y obtiene un hash de 256 bits con PBKDF2WithHmacSHA256 y 120 000 iteraciones. Al iniciar sesión se vuelve a derivar el hash con la misma sal y se compara con MessageDigest.isEqual. Esta implementación sirve para la autenticación local del laboratorio.

Cada compra se asocia al ID del usuario. El historial solo muestra las compras de la sesión actual. Para reseñar, el modelo comprueba que exista una compra confirmada de ese usuario que contenga el producto. La calificación debe ser de 1 a 5, el texto debe tener contenido y se permite una reseña por usuario y producto. Las reseñas pueden consultarse junto con su promedio.

La sesión se exige antes de manejar el carrito. Para entrar con otra cuenta se debe cerrar la sesión anterior; al cerrarla se limpian el carrito, el descuento y el envío. Esto evita que otro usuario vea un carrito pendiente de la cuenta anterior. El catálogo y las reseñas publicadas pueden consultarse sin sesión.

Explicación de los archivos de los ejercicios

Archivo: lab6_ejercicios1_2/src/modelo/Producto.java

```
package modelo;

import java.math.BigDecimal;

public class Producto {
  private int id;
  private String nombre;
  private BigDecimal precio;

  public Producto(int id, String nombre, BigDecimal precio)
  {
      this.id = id;
      this.nombre = nombre;
      this.precio = precio;
  }

  public int getId() {
      return id;
  }

  public String getNombre() {
      return nombre;
  }

  public BigDecimal getPrecio() {
      return precio;
```

```
    }
}
```

Explicación del archivo: Producto.java

Representa un artículo del catálogo mediante id, nombre y precio. El constructor recibe los tres valores y los getters permiten consultarlos. No contiene setters, por lo que el precio que se usa en una línea del carrito no cambia por una edición posterior del catálogo.

El precio es BigDecimal para mantener aritmética decimal en los importes. TiendaModelo valida y ajusta el precio antes de crear Producto. Esta clase conserva los datos del artículo; no conoce usuarios, menús ni compras.

Archivo: lab6_ejercicios1_2/src/modelo/ItemCarrito.java

```
package modelo;

import java.math.BigDecimal;

public class ItemCarrito {
  private Producto producto;
  private int cantidad;

  public ItemCarrito(Producto producto, int cantidad)
  {
      this.producto = producto;
      this.cantidad = cantidad;
  }

  public Producto getProducto() {
      return producto;
  }

  public int getCantidad() {
      return cantidad;
  }

  public void setCantidad(int cantidad) {
      this.cantidad = cantidad;
  }

  public BigDecimal getSubtotal()
  {
      return producto.getPrecio().multiply(BigDecimal.valueOf(cantidad));
  }
}
```

Explicación del archivo: ItemCarrito.java

Agrupa un Producto con su cantidad. Los getters permiten mostrar la línea y setCantidad permite acumular unidades de un producto existente. getSubtotal multiplica el precio por BigDecimal.valueOf(cantidad).

Usé una clase para la línea porque el producto del catálogo y la cantidad seleccionada tienen responsabilidades diferentes. Así se pueden conservar dos teclados en una compra y uno en un carrito nuevo sin modificar el Producto.

Archivo: lab6_ejercicios1_2/src/modelo/Compra.java

```
package modelo;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class Compra {
  private int id;
  private int usuarioId;
  private LocalDateTime fecha;
  private List<ItemCarrito> productos;
  private BigDecimal subtotal;
  private BigDecimal descuento;
  private BigDecimal envio;

  public Compra(int id, int usuarioId, List<ItemCarrito> carrito,
          BigDecimal subtotal, BigDecimal descuento, BigDecimal envio)
  {
      this.id = id;
      this.usuarioId = usuarioId;
      fecha = LocalDateTime.now();
      productos = new ArrayList<>();
      for(ItemCarrito item: carrito)
      {
          productos.add(new ItemCarrito(item.getProducto(), item.getCantidad()));
      }
      this.subtotal = subtotal;
      this.descuento = descuento;
      this.envio = envio;
  }

  public int getId() {
      return id;
  }

  public int getUsuarioId() {
      return usuarioId;
  }

  public LocalDateTime getFecha() {
      return fecha;
  }

  public List<ItemCarrito> getProductos() {
```

```
        return new ArrayList<>(productos);
    }

    public BigDecimal getSubtotal() {
        return subtotal;
    }

    public BigDecimal getDescuento() {
        return descuento;
    }

    public BigDecimal getEnvio() {
        return envio;
    }

    public BigDecimal getTotal()
    {
        return subtotal.subtract(descuento).add(envio);
    }
}
```

Explicación del archivo: Compra.java

Guarda id, usuarioId, fecha, productos, subtotal, descuento y envío. El constructor toma LocalDateTime.now y crea un ItemCarrito nuevo por cada línea recibida. Esa copia evita que cambiar la cantidad del carrito modifique una compra ya confirmada.

getTotal usa los importes guardados al comprar y calcula subtotal menos descuento más envío. getProductos devuelve una nueva lista para que retirar una línea de la lista retornada no retire registros de la compra. Los getters entregan el resto de datos a la vista. No se vuelve a calcular el historial con el descuento del carrito actual.

Archivo: lab6_ejercicios1_2/src/modelo/Usuario.java

```
package modelo;

import java.security.MessageDigest;
import java.security.SecureRandom;
import java.security.GeneralSecurityException;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;

public class Usuario {
  private int id;
  private String nombre;
  private String correo;
  private byte[] sal;
  private byte[] claveHash;

  public Usuario(int id, String nombre, String correo, String clave)
  {
      this.id = id;
```

```
        this.nombre = nombre;
        this.correo = correo;
        sal = new byte[16];
        new SecureRandom().nextBytes(sal);
        claveHash = generarHash(clave);
    }

    private byte[] generarHash(String clave)
    {
        PBEKeySpec datos = new PBEKeySpec(clave.toCharArray(), sal, 120000, 256);
        try
        {
            return
SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(datos).getEncode
d();
        }
        catch(GeneralSecurityException e)
        {
            throw new IllegalStateException("No se pudo procesar la contraseña.", e);
        }
        finally
        {
            datos.clearPassword();
        }
    }

    public boolean verificarClave(String clave)
    {
        return MessageDigest.isEqual(claveHash, generarHash(clave));
    }

    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getCorreo() {
        return correo;
    }
}
```

Explicación del archivo: Usuario.java

Representa una cuenta con id, nombre, correo, sal y claveHash. El constructor crea la sal con SecureRandom y llama a generarHash. Ese método usa PBEKeySpec, SecretKeyFactory y PBKDF2WithHmacSHA256; en finally limpia la contraseña temporal del objeto PBEKeySpec.

verificarClave recibe la contraseña escrita, deriva su hash y lo compara con el almacenado. Los getters exponen únicamente id, nombre y correo. No existe un getter para la contraseña, la sal o el hash. El modelo de usuarios decide si una cuenta puede registrarse o iniciar sesión; Usuario realiza la comprobación de la clave.

Archivo: lab6_ejercicios1_2/src/modelo/UsuarioModelo.java

```
package modelo;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

public class UsuarioModelo {
  private List<Usuario> usuarios;
  private Usuario usuarioActual;

  public UsuarioModelo()
  {
      usuarios = new ArrayList<>();
  }

  public void registrar(String nombre, String correo, String clave)
  {
      nombre = nombre.trim();
      correo = correo.trim().toLowerCase(Locale.ROOT);
      if(nombre.isEmpty() || !correo.matches("[^\\s@]+@[^\\s@]+\\.[^\\s@]+"))
      {
          throw new IllegalArgumentException("Nombre o correo inválido.");
      }
      if(clave.trim().length() < 8)
      {
          throw new IllegalArgumentException("La contraseña debe tener al menos 8
caracteres.");
      }
      for(Usuario usuario: usuarios)
      {
          if(usuario.getCorreo().equals(correo))
          {
              throw new IllegalArgumentException("El correo ya está registrado.");
          }
      }
      usuarios.add(new Usuario(usuarios.size() + 1, nombre, correo, clave));
  }

  public void iniciarSesion(String correo, String clave)
  {
      correo = correo.trim().toLowerCase(Locale.ROOT);
      for(Usuario usuario: usuarios)
      {
          if(usuario.getCorreo().equals(correo) && usuario.verificarClave(clave))
```

```
            {
                usuarioActual = usuario;
                return;
            }
        }
        throw new IllegalArgumentException("Correo o contraseña incorrectos.");
    }

    public void cerrarSesion()
    {
        usuarioActual = null;
    }

    public Usuario getUsuarioActual()
    {
        return usuarioActual;
    }

    public Usuario exigirSesion()
    {
        if(usuarioActual == null)
        {
            throw new IllegalArgumentException("Primero debes iniciar sesión.");
        }
        return usuarioActual;
    }
}
```

Explicación del archivo: UsuarioModelo.java

Administra la lista de cuentas y la referencia usuarioActual. registrar limpia nombre y correo, normaliza el correo con Locale.ROOT, valida sus datos y recorre la lista para rechazar duplicados antes de crear Usuario. Los IDs se obtienen del número de cuentas porque este sistema no elimina usuarios.

iniciarSesion busca el correo normalizado y verifica la contraseña; solo si ambos son correctos establece usuarioActual. Una credencial incorrecta genera una excepción. cerrarSesion asigna null. getUsuarioActual permite consultar la sesión y exigirSesion devuelve el usuario o rechaza una operación protegida. El modelo no imprime información ni maneja el carrito.

Archivo: lab6_ejercicios1_2/src/modelo/Resena.java

```
package modelo;

public class Resena {
  private int productoId;
  private int usuarioId;
  private String nombreUsuario;
  private int calificacion;
  private String comentario;

  public Resena(int productoId, Usuario usuario, int calificacion, String comentario)
```

```
    {
        this.productoId = productoId;
        usuarioId = usuario.getId();
        nombreUsuario = usuario.getNombre();
        this.calificacion = calificacion;
        this.comentario = comentario;
    }

    public int getProductoId() {
        return productoId;
    }

    public int getUsuarioId() {
        return usuarioId;
    }

    public String getNombreUsuario() {
        return nombreUsuario;
    }

    public int getCalificacion() {
        return calificacion;
    }

    public String getComentario() {
        return comentario;
    }
}
```

Explicación del archivo: Resena.java

Conserva productoId, usuarioId, nombreUsuario, calificacion y comentario. El constructor obtiene la identidad del Usuario que reseña. Esto relaciona la reseña con la cuenta y permite mostrar el nombre del comprador sin recibirlo como texto libre desde la consola.

La clase contiene getters y no modifica compras ni sesiones. TiendaModelo verifica previamente la compra, el rango de calificación y la ausencia de una reseña anterior. Luego crea Resena con el texto limpio.

Archivo: lab6_ejercicios1_2/src/modelo/TiendaModelo.java

```
package modelo;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.List;

public class TiendaModelo {
  private List<Producto> productos;
  private List<ItemCarrito> carrito;
  private List<Compra> compras;
```

```
private List<Resena> resenas;
private int porcentajeDescuento;
private String zonaEnvio;

public TiendaModelo()
{
    productos = new ArrayList<>();
    carrito = new ArrayList<>();
    compras = new ArrayList<>();
    resenas = new ArrayList<>();
    zonaEnvio = "RECOJO";
}

public void agregarProducto(String nombre, BigDecimal precio)
{
    if(nombre.trim().isEmpty() || precio.compareTo(BigDecimal.ZERO) <= 0
            || precio.compareTo(new BigDecimal("1000000")) > 0)
    {
        throw new IllegalArgumentException("Nombre vacío o precio fuera de rango.");
    }
    precio = precio.setScale(2, RoundingMode.UNNECESSARY);
    productos.add(new Producto(productos.size() + 1, nombre.trim(), precio));
}

public Producto buscarProducto(int id)
{
    for(Producto producto: productos)
    {
        if(producto.getId() == id)
        {
            return producto;
        }
    }
    throw new IllegalArgumentException("Producto no encontrado.");
}

public void agregarAlCarrito(int id, int cantidad)
{
    Producto producto = buscarProducto(id);
    if(cantidad < 1 || cantidad > 1000)
    {
        throw new IllegalArgumentException("La cantidad debe estar entre 1 y 1000.");
    }
    for(ItemCarrito item: carrito)
    {
        if(item.getProducto().getId() == id)
        {
            if(item.getCantidad() + cantidad > 1000)
            {
                throw new IllegalArgumentException("Máximo 1000 unidades por producto.");
            }
```

```
                item.setCantidad(item.getCantidad() + cantidad);
                return;
            }
        }
        carrito.add(new ItemCarrito(producto, cantidad));
    }

    public void eliminarDelCarrito(int id)
    {
        for(int i = 0; i < carrito.size(); i++)
        {
            if(carrito.get(i).getProducto().getId() == id)
            {
                carrito.remove(i);
                return;
            }
        }
        throw new IllegalArgumentException("El producto no está en el carrito.");
    }

    public void aplicarDescuento(int porcentaje)
    {
        if(porcentaje < 0 || porcentaje > 100)
        {
            throw new IllegalArgumentException("El descuento debe estar entre 0 y 100.");
        }
        porcentajeDescuento = porcentaje;
    }

    public void seleccionarEnvio(String zona)
    {
        zona = zona.trim().toUpperCase(java.util.Locale.ROOT);
        if(!zona.equals("RECOJO") && !zona.equals("LOCAL") && !
zona.equals("NACIONAL"))
        {
            throw new IllegalArgumentException("Elige RECOJO, LOCAL o NACIONAL.");
        }
        zonaEnvio = zona;
    }

    public BigDecimal calcularSubtotal()
    {
        BigDecimal subtotal = BigDecimal.ZERO;
        for(ItemCarrito item: carrito)
        {
            subtotal = subtotal.add(item.getSubtotal());
        }
        return subtotal.setScale(2);
    }

    public BigDecimal calcularDescuento()
```

```
{
    return calcularSubtotal().multiply(BigDecimal.valueOf(porcentajeDescuento))
            .divide(new BigDecimal("100"), 2, RoundingMode.HALF_UP);
}

public BigDecimal calcularEnvio()
{
    if(carrito.isEmpty() || zonaEnvio.equals("RECOJO"))
    {
        return new BigDecimal("0.00");
    }
    if(zonaEnvio.equals("LOCAL"))
    {
        return new BigDecimal("10.00");
    }
    return new BigDecimal("25.00");
}

public BigDecimal calcularTotal()
{
    return calcularSubtotal().subtract(calcularDescuento()).add(calcularEnvio());
}

public Compra realizarCompra(Usuario usuario)
{
    if(usuario == null)
    {
        throw new IllegalArgumentException("Primero debes iniciar sesión.");
    }
    if(carrito.isEmpty())
    {
        throw new IllegalArgumentException("El carrito está vacío.");
    }
    Compra compra = new Compra(compras.size() + 1, usuario.getId(), carrito,
            calcularSubtotal(), calcularDescuento(), calcularEnvio());
    compras.add(compra);
    vaciarCarrito();
    return compra;
}

public void vaciarCarrito()
{
    carrito.clear();
    porcentajeDescuento = 0;
    zonaEnvio = "RECOJO";
}

public List<Compra> historialUsuario(Usuario usuario)
{
    List<Compra> historial = new ArrayList<>();
    for(Compra compra: compras)
```

```
        {
            if(compra.getUsuarioId() == usuario.getId())
            {
                historial.add(compra);
            }
        }
        return historial;
    }

    public boolean productoComprado(Usuario usuario, int id)
    {
        for(Compra compra: historialUsuario(usuario))
        {
            for(ItemCarrito item: compra.getProductos())
            {
                if(item.getProducto().getId() == id)
                {
                    return true;
                }
            }
        }
        return false;
    }

    public void agregarResena(Usuario usuario, int id, int calificacion, String comentario)
    {
        buscarProducto(id);
        if(usuario == null || !productoComprado(usuario, id))
        {
            throw new IllegalArgumentException("Solo puedes reseñar productos que hayas
comprado.");
        }
        if(calificacion < 1 || calificacion > 5 || comentario.trim().isEmpty())
        {
            throw new IllegalArgumentException("Ingresa una calificación de 1 a 5 y una reseña.");
        }
        for(Resena resena: resenas)
        {
            if(resena.getProductoId() == id && resena.getUsuarioId() == usuario.getId())
            {
                throw new IllegalArgumentException("Ya reseñaste este producto.");
            }
        }
        resenas.add(new Resena(id, usuario, calificacion, comentario.trim()));
    }

    public List<Resena> resenasProducto(int id)
    {
        buscarProducto(id);
        List<Resena> resultado = new ArrayList<>();
        for(Resena resena: resenas)
```

```
        {
            if(resena.getProductoId() == id)
            {
                resultado.add(resena);
            }
        }
        return resultado;
    }

    public double promedioProducto(int id)
    {
        List<Resena> resultado = resenasProducto(id);
        int suma = 0;
        for(Resena resena: resultado)
        {
            suma += resena.getCalificacion();
        }
        if(resultado.isEmpty())
        {
            return 0;
        }
        return (double) suma / resultado.size();
    }

    public List<Producto> getProductos() {
        return new ArrayList<>(productos);
    }

    public List<ItemCarrito> getCarrito() {
        return new ArrayList<>(carrito);
    }
}
```

Explicación del archivo: TiendaModelo.java

Centraliza el catálogo, el carrito, las compras y las reseñas en cuatro ArrayList. También guarda porcentajeDescuento y zonaEnvio. agregarProducto valida nombre y precio y asigna un ID. buscarProducto recorre el catálogo y rechaza un ID inexistente. agregarAlCarrito valida cantidad, acumula unidades si la línea existe y crea una línea nueva en caso contrario. eliminarDelCarrito busca por ID y retira la línea completa.

aplicarDescuento acepta porcentajes entre 0 y 100 y reemplaza el anterior, sin acumular descuentos. seleccionarEnvio normaliza y valida la zona. calcularSubtotal suma los subtotales; calcularDescuento aplica el porcentaje y el redondeo; calcularEnvio devuelve la tarifa elegida o cero si no hay productos; calcularTotal reúne los tres importes.

realizarCompra verifica usuario y carrito, construye la compra con importes y líneas actuales, la añade a compras y llama a vaciarCarrito. Este último reinicia también descuento y envío. historialUsuario filtra compras por usuarioId. productoComprado recorre ese historial y las líneas de cada compra para comprobar que el producto realmente se compró.

agregarResena busca el producto, verifica al comprador, valida texto y calificación y rechaza la combinación repetida de usuario y producto. resenasProducto devuelve las reseñas del artículo y promedioProducto divide la suma entre la cantidad, devolviendo cero si no existen reseñas. getProductos y getCarrito retornan listas nuevas. Ninguna función del modelo lee el teclado ni imprime mensajes.

Archivo: lab6_ejercicios1_2/src/vista/TiendaVista.java

```
package vista;

import java.math.BigDecimal;
import java.time.format.DateTimeFormatter;
import java.util.List;
import java.util.Locale;
import java.util.Scanner;
import modelo.Compra;
import modelo.ItemCarrito;
import modelo.Producto;
import modelo.Resena;

public class TiendaVista {
  private Scanner scanner;

  public TiendaVista(Scanner scanner)
  {
      this.scanner = scanner;
  }

  public String solicitarTexto(String mensaje)
  {
      System.out.print(mensaje);
      if(!scanner.hasNextLine())
      {
          throw new IllegalArgumentException("Entrada incompleta.");
      }
      return scanner.nextLine();
  }

  public String solicitarOpcion()
  {
      System.out.print("Opción: ");
      if(!scanner.hasNextLine())
      {
          return "0";
      }
      return scanner.nextLine();
  }

  public int solicitarNumero(String mensaje)
  {
      try
```

```
    {
        return Integer.parseInt(solicitarTexto(mensaje).trim());
    }
    catch(NumberFormatException e)
    {
        throw new IllegalArgumentException("Debes ingresar un número entero.");
    }
}

public BigDecimal solicitarPrecio()
{
    try
    {
        return new BigDecimal(solicitarTexto("Precio S/: ").trim().replace(',', '.'));
    }
    catch(NumberFormatException e)
    {
        throw new IllegalArgumentException("Precio inválido.");
    }
}

public void mostrarMenu()
{
    mostrarMensaje("\nCARRITO DE COMPRAS");
    mostrarMensaje("1. Agregar producto");
    mostrarMensaje("2. Listar productos");
    mostrarMensaje("3. Agregar al carrito");
    mostrarMensaje("4. Ver carrito");
    mostrarMensaje("5. Eliminar del carrito");
    mostrarMensaje("6. Aplicar descuento");
    mostrarMensaje("7. Calcular envío");
    mostrarMensaje("8. Realizar compra");
    mostrarMensaje("9. Historial de compras");
    mostrarMensaje("10. Registrarse");
    mostrarMensaje("11. Iniciar sesión");
    mostrarMensaje("12. Cerrar sesión");
    mostrarMensaje("13. Agregar reseña y calificación");
    mostrarMensaje("14. Ver reseñas");
    mostrarMensaje("0. Salir");
}

public void mostrarProductos(List<Producto> productos)
{
    if(productos.isEmpty())
    {
        mostrarMensaje("No hay productos.");
    }
    for(Producto producto: productos)
    {
        mostrarMensaje(producto.getId() + " | " + producto.getNombre()
                + " | S/ " + producto.getPrecio().toPlainString());
```

```
    }
}

public void mostrarItems(List<ItemCarrito> items)
{
    if(items.isEmpty())
    {
        mostrarMensaje("El carrito está vacío.");
    }
    for(ItemCarrito item: items)
    {
        mostrarMensaje(item.getProducto().getId() + " | " + item.getProducto().getNombre()
                + " | Cantidad: " + item.getCantidad()
                + " | S/ " + item.getSubtotal().toPlainString());
    }
}

public void mostrarTotales(BigDecimal subtotal, BigDecimal descuento, BigDecimal envio,
BigDecimal total)
{
    mostrarMensaje("Subtotal: S/ " + subtotal.toPlainString());
    mostrarMensaje("Descuento: S/ " + descuento.toPlainString());
    mostrarMensaje("Envío: S/ " + envio.toPlainString());
    mostrarMensaje("Total: S/ " + total.toPlainString());
}

public void mostrarCompra(Compra compra)
{
    mostrarMensaje("Compra #" + compra.getId() + " | "
            + compra.getFecha().format(DateTimeFormatter.ofPattern("dd/MM/yyyy
HH:mm")));
    mostrarItems(compra.getProductos());
    mostrarTotales(compra.getSubtotal(), compra.getDescuento(), compra.getEnvio(),
compra.getTotal());
}

public void mostrarHistorial(List<Compra> compras)
{
    if(compras.isEmpty())
    {
        mostrarMensaje("No tienes compras registradas.");
    }
    for(Compra compra: compras)
    {
        mostrarCompra(compra);
    }
}

public void mostrarResenas(List<Resena> resenas, double promedio)
{
    if(resenas.isEmpty())
```

```
        {
            mostrarMensaje("El producto no tiene reseñas.");
            return;
        }
        for(Resena resena: resenas)
        {
            mostrarMensaje(resena.getNombreUsuario() + " | " + resena.getCalificacion()
                    + "/5 | " + resena.getComentario());
        }
        mostrarMensaje(String.format(Locale.ROOT, "Promedio: %.2f/5", promedio));
    }

    public void mostrarMensaje(String mensaje)
    {
        System.out.println(mensaje);
    }
}
```

Explicación del archivo: TiendaVista.java

Recibe un Scanner desde Main. solicitarTexto lee una línea, solicitarNumero convierte un entero y solicitarPrecio crea un BigDecimal aceptando punto o coma decimal. solicitarOpcion devuelve 0 cuando termina la entrada para cerrar el menú. Los errores de formato se expresan mediante excepciones que el controlador muestra.

mostrarMenu enumera las 14 operaciones y la salida. mostrarProductos presenta ID, nombre y precio; mostrarItems presenta cantidad y subtotal por línea. mostrarTotales recibe importes ya calculados. mostrarCompra muestra ID, fecha, líneas e importes, y mostrarHistorial aplica esa presentación a todas las compras recibidas. mostrarResenas presenta autor, nota, texto y promedio, formateado con dos decimales. La vista decide cómo se ve la información y no quién puede comprar o reseñar.

Archivo: lab6_ejercicios1_2/src/vista/UsuarioVista.java

```
package vista;

import java.util.Scanner;
import modelo.Usuario;

public class UsuarioVista {
  private Scanner scanner;

  public UsuarioVista(Scanner scanner)
  {
      this.scanner = scanner;
  }

  public String solicitarTexto(String mensaje)
  {
      System.out.print(mensaje);
      if(!scanner.hasNextLine())
```

```
        {
            throw new IllegalArgumentException("Entrada incompleta.");
        }
        return scanner.nextLine();
    }

    public void mostrarUsuario(Usuario usuario)
    {
        if(usuario == null)
        {
            mostrarMensaje("Usuario: sin sesión");
        }
        else
        {
            mostrarMensaje("Usuario: " + usuario.getNombre());
        }
    }

    public void mostrarMensaje(String mensaje)
    {
        System.out.println(mensaje);
    }
}
```

Explicación del archivo: UsuarioVista.java

Comparte el Scanner de TiendaVista para evitar que dos lectores distintos consuman System.in. solicitarTexto presenta los campos de registro y acceso. mostrarUsuario indica si hay sesión y, cuando existe, muestra el nombre. mostrarMensaje presenta confirmaciones.

Separé esta vista para que UsuarioControlador no tenga que manejar la presentación de productos ni las compras. La contraseña se solicita por consola dentro de la práctica; Scanner no oculta los caracteres escritos. Los ejemplos de ejecución usan credenciales ficticias.

Archivo: lab6_ejercicios1_2/src/controlador/UsuarioControlador.java

```
package controlador;

import modelo.UsuarioModelo;
import vista.UsuarioVista;

public class UsuarioControlador {
  private UsuarioModelo modelo;
  private UsuarioVista vista;

  public UsuarioControlador(UsuarioModelo modelo, UsuarioVista vista)
  {
      this.modelo = modelo;
      this.vista = vista;
  }

  public void registrar()
```

```
    {
        String nombre = vista.solicitarTexto("Nombre: ");
        String correo = vista.solicitarTexto("Correo: ");
        String clave = vista.solicitarTexto("Contraseña: ");
        modelo.registrar(nombre, correo, clave);
        vista.mostrarMensaje("Usuario registrado.");
    }

    public void iniciarSesion()
    {
        String correo = vista.solicitarTexto("Correo: ");
        String clave = vista.solicitarTexto("Contraseña: ");
        modelo.iniciarSesion(correo, clave);
        vista.mostrarMensaje("Sesión iniciada.");
    }

    public void cerrarSesion()
    {
        modelo.cerrarSesion();
        vista.mostrarMensaje("Sesión cerrada.");
    }

    public void mostrarUsuario()
    {
        vista.mostrarUsuario(modelo.getUsuarioActual());
    }
}
```

Explicación del archivo: UsuarioControlador.java

Coordina UsuarioModelo y UsuarioVista. registrar solicita nombre, correo y clave y llama al modelo. iniciarSesion solicita credenciales y muestra la confirmación únicamente después del acceso correcto. cerrarSesion elimina la sesión en el modelo y presenta el resultado. mostrarUsuario delega la presentación del usuario actual.

La validación del correo y la clave permanece en UsuarioModelo y Usuario. Las excepciones llegan a TiendaControlador, que las muestra dentro del menú general. Esto mantiene las operaciones de autenticación separadas de las operaciones comerciales.

Archivo: lab6_ejercicios1_2/src/controlador/TiendaControlador.java

```
package controlador;

import modelo.Compra;
import modelo.TiendaModelo;
import modelo.UsuarioModelo;
import vista.TiendaVista;

public class TiendaControlador {
  private TiendaModelo modelo;
  private TiendaVista vista;
  private UsuarioModelo usuarios;
```

```
private UsuarioControlador usuarioControlador;

public TiendaControlador(TiendaModelo modelo, TiendaVista vista,
        UsuarioModelo usuarios, UsuarioControlador usuarioControlador)
{
    this.modelo = modelo;
    this.vista = vista;
    this.usuarios = usuarios;
    this.usuarioControlador = usuarioControlador;
}

public void mostrarCarrito()
{
    vista.mostrarItems(modelo.getCarrito());
    vista.mostrarTotales(modelo.calcularSubtotal(), modelo.calcularDescuento(),
            modelo.calcularEnvio(), modelo.calcularTotal());
}

public void comprar()
{
    usuarios.exigirSesion();
    if(modelo.getCarrito().isEmpty())
    {
        throw new IllegalArgumentException("El carrito está vacío.");
    }
    mostrarCarrito();
    String confirmacion = vista.solicitarTexto("Confirmar compra (S/N): ");
    if(!confirmacion.trim().equalsIgnoreCase("S"))
    {
        vista.mostrarMensaje("Compra cancelada.");
        return;
    }
    Compra compra = modelo.realizarCompra(usuarios.exigirSesion());
    vista.mostrarMensaje("Compra realizada.");
    vista.mostrarCompra(compra);
}

public void iniciar()
{
    String opcion;
    do
    {
        vista.mostrarMenu();
        usuarioControlador.mostrarUsuario();
        opcion = vista.solicitarOpcion();
        try
        {
            switch(opcion)
            {
            case "1":
                String nombre = vista.solicitarTexto("Nombre del producto: ");
```

```
                modelo.agregarProducto(nombre, vista.solicitarPrecio());
                vista.mostrarMensaje("Producto agregado.");
                break;
            case "2":
                vista.mostrarProductos(modelo.getProductos());
                break;
            case "3":
                usuarios.exigirSesion();
                int id = vista.solicitarNumero("ID del producto: ");
                modelo.agregarAlCarrito(id, vista.solicitarNumero("Cantidad: "));
                vista.mostrarMensaje("Producto agregado al carrito.");
                break;
            case "4":
                usuarios.exigirSesion();
                mostrarCarrito();
                break;
            case "5":
                usuarios.exigirSesion();
                modelo.eliminarDelCarrito(vista.solicitarNumero("ID a eliminar: "));
                vista.mostrarMensaje("Producto eliminado del carrito.");
                break;
            case "6":
                usuarios.exigirSesion();
                modelo.aplicarDescuento(vista.solicitarNumero("Descuento en porcentaje: "));
                mostrarCarrito();
                break;
            case "7":
                usuarios.exigirSesion();
                modelo.seleccionarEnvio(vista.solicitarTexto("Zona
(RECOJO/LOCAL/NACIONAL): "));
                mostrarCarrito();
                break;
            case "8":
                comprar();
                break;
            case "9":
                vista.mostrarHistorial(modelo.historialUsuario(usuarios.exigirSesion()));
                break;
            case "10":
                usuarioControlador.registrar();
                break;
            case "11":
                if(usuarios.getUsuarioActual() != null)
                {
                    throw new IllegalArgumentException("Cierra la sesión actual primero.");
                }
                usuarioControlador.iniciarSesion();
                modelo.vaciarCarrito();
                break;
            case "12":
                usuarioControlador.cerrarSesion();
```

```
                    modelo.vaciarCarrito();
                    break;
                case "13":
                    usuarios.exigirSesion();
                    int productoId = vista.solicitarNumero("ID a reseñar: ");
                    int calificacion = vista.solicitarNumero("Calificación (1 a 5): ");
                    String comentario = vista.solicitarTexto("Reseña: ");
                    modelo.agregarResena(usuarios.exigirSesion(), productoId, calificacion,
comentario);
                    vista.mostrarMensaje("Reseña registrada.");
                    break;
                case "14":
                    int producto = vista.solicitarNumero("ID del producto: ");
                    vista.mostrarResenas(modelo.resenasProducto(producto),
modelo.promedioProducto(producto));
                    break;
                case "0":
                    vista.mostrarMensaje("Saliendo...");
                    break;
                default:
                    vista.mostrarMensaje("Opción no válida.");
                }
            }
            catch(IllegalArgumentException e)
            {
                vista.mostrarMensaje(e.getMessage());
            }
            catch(ArithmeticException e)
            {
                vista.mostrarMensaje("Usa un precio con máximo dos decimales.");
            }
        } while(!opcion.equals("0"));
    }
}
```

Explicación del archivo: TiendaControlador.java

Recibe TiendaModelo, TiendaVista, UsuarioModelo y UsuarioControlador. iniciar usa un do while y un switch para coordinar las 14 opciones. Antes de manejar un carrito, comprar, ver historial o reseñar llama a exigirSesion. El registro y acceso se delegan a UsuarioControlador; después de un acceso correcto se limpia el carrito.

mostrarCarrito entrega a la vista las líneas y los importes calculados por el modelo. comprar valida sesión y contenido, muestra el total y solicita S o N. Solo S llama a realizarCompra y muestra el comprobante local. Al cerrar sesión se llama a vaciarCarrito. La opción de reseñar solicita ID, nota y texto; la decisión de permitirla se realiza en TiendaModelo.

Los errores de entrada o negocio se capturan como IllegalArgumentException. ArithmeticException cubre los precios con decimales no admitidos y muestra un mensaje específico. La compra cancelada no registra una compra ni vacía el carrito. Los casos incluyen break y 0 finaliza el ciclo.

Archivo: lab6_ejercicios1_2/src/principal/Main.java

```
package principal;

import java.util.Scanner;
import modelo.TiendaModelo;
import modelo.UsuarioModelo;
import vista.TiendaVista;
import vista.UsuarioVista;
import controlador.TiendaControlador;
import controlador.UsuarioControlador;

public class Main {
  public static void main(String[] args)
  {
      Scanner scanner = new Scanner(System.in);
      TiendaModelo modelo = new TiendaModelo();
      UsuarioModelo usuarios = new UsuarioModelo();
      TiendaVista vista = new TiendaVista(scanner);
      UsuarioVista usuarioVista = new UsuarioVista(scanner);
      UsuarioControlador usuarioControlador = new UsuarioControlador(usuarios, usuarioVista);
      TiendaControlador controlador = new TiendaControlador(modelo, vista, usuarios,
usuarioControlador);
      controlador.iniciar();
      scanner.close();
  }
}
```

Explicación del archivo: Main.java

Crea un solo Scanner, los dos modelos y las dos vistas. Después construye UsuarioControlador y lo entrega a TiendaControlador junto con sus demás dependencias. Llama a iniciar y cierra el Scanner al terminar.

El constructor de cada componente recibe los objetos que necesita, sin utilizar variables globales ni Singleton. Ambas vistas reciben la misma entrada de consola. La clase principal permite ver la composición de las dos partes MVC antes de iniciar la interacción.

Archivo: lab6_ejercicios1_2/src/module-info.java

```
module laboratorio.tienda {
}
```

Explicación del archivo: module-info.java

Declara laboratorio.tienda. Las colecciones, BigDecimal, LocalDateTime y las clases criptográficas utilizadas forman parte de java.base. Por eso el módulo no requiere bibliotecas externas. El código puede compilarse con JDK 17 y ejecutarse desde Eclipse o mediante los comandos del anexo.

Archivos de configuración del proyecto de ejercicios

Archivo: lab6_ejercicios1_2/.project

```
<?xml version="1.0" encoding="UTF-8"?>
<projectDescription>
  <name>lab6_ejercicios1_2</name>
  <comment></comment>
  <projects>
  </projects>
  <buildSpec>
      <buildCommand>
          <name>org.eclipse.jdt.core.javabuilder</name>
          <arguments>
          </arguments>
      </buildCommand>
  </buildSpec>
  <natures>
      <nature>org.eclipse.jdt.core.javanature</nature>
  </natures>
</projectDescription>
```

Explicación del archivo: .project

Identifica el proyecto de Eclipse mediante su nombre. buildSpec utiliza org.eclipse.jdt.core.javabuilder para compilar Java y natures declara org.eclipse.jdt.core.javanature para que el IDE lo reconozca como proyecto Java. Se conserva una configuración simple sin dependencias ni constructores adicionales.

Archivo: lab6_ejercicios1_2/.classpath

```
<?xml version="1.0" encoding="UTF-8"?>
<classpath>
  <classpathentry kind="con"
path="org.eclipse.jdt.launching.JRE_CONTAINER/org.eclipse.jdt.internal.debug.ui.launcher.Sta
ndardVMType/JavaSE-17">
      <attributes>
          <attribute name="module" value="true"/>
      </attributes>
  </classpathentry>
  <classpathentry kind="src" path="src"/>
  <classpathentry kind="output" path="bin"/>
</classpath>
```

Explicación del archivo: .classpath

Declara src como carpeta de entrada, el contenedor de JavaSE-17 como biblioteca del sistema y bin como carpeta de salida. Eclipse puede compilar el proyecto a partir de estas rutas. Cada proyecto mantiene su propia salida para evitar mezclar las dos clases principal.Main.

Archivo: lab6_ejercicios1_2/.settings/org.eclipse.core.resources.prefs

```
eclipse.preferences.version=1
encoding/<project>=UTF-8
```

Explicación del archivo: org.eclipse.core.resources.prefs

Establece UTF-8 como codificación del proyecto. Es necesaria para conservar las tildes, la ñ y los mensajes de consola en español. eclipse.preferences.version identifica la versión del archivo de preferencias.

Archivo: lab6_ejercicios1_2/.settings/org.eclipse.jdt.core.prefs

```
eclipse.preferences.version=1
org.eclipse.jdt.core.compiler.codegen.targetPlatform=17
org.eclipse.jdt.core.compiler.codegen.unusedLocal=preserve
org.eclipse.jdt.core.compiler.compliance=17
org.eclipse.jdt.core.compiler.debug.lineNumber=generate
org.eclipse.jdt.core.compiler.debug.localVariable=generate
org.eclipse.jdt.core.compiler.debug.sourceFile=generate
org.eclipse.jdt.core.compiler.problem.enablePreviewFeatures=disabled
org.eclipse.jdt.core.compiler.problem.reportPreviewFeatures=warning
org.eclipse.jdt.core.compiler.release=enabled
org.eclipse.jdt.core.compiler.source=17
```

Explicación del archivo: org.eclipse.jdt.core.prefs

Configura Java 17 como nivel de fuente, compatibilidad y destino de las clases. Desactiva las características preview para que el código pueda ejecutarse con una instalación normal de JDK 17. Se usa el mismo nivel en los dos proyectos y en la compilación de verificación.

Archivo: lab6_ejercicios1_2/fuentes.txt

```
src/controlador/TiendaControlador.java
src/controlador/UsuarioControlador.java
src/modelo/Compra.java
src/modelo/ItemCarrito.java
src/modelo/Producto.java
src/modelo/Resena.java
src/modelo/TiendaModelo.java
src/modelo/Usuario.java
src/modelo/UsuarioModelo.java
src/module-info.java
src/principal/Main.java
src/vista/TiendaVista.java
src/vista/UsuarioVista.java
```

Explicación del archivo: fuentes.txt

Enumera todos los archivos Java del proyecto mediante rutas relativas. El compilador acepta @fuentes.txt y lee cada ruta como un argumento, de modo que el comando de compilación incluye

modelos, vistas, controladores, Main y module-info sin depender de un comodín específico del sistema operativo.

Opciones de los ejercicios

Opción	Operación
1 y 2	Agregar y listar productos.
3 a 5	Agregar, ver y eliminar productos del carrito.
6 y 7	Aplicar descuento y seleccionar envío.
8 y 9	Confirmar compra y consultar el historial propio.
10 a 12	Registrar cuenta, iniciar sesión y cerrar sesión.
13 y 14	Añadir una reseña y consultar reseñas con promedio.
0	Salir.

Compilación de los ejercicios

Compilación del carrito con usuarios y reseñas

La compilación real genera las clases de las dos partes MVC. UsuarioModelo y UsuarioControlador se compilan junto con los componentes de Tienda y con las clases que representan compras y reseñas.

```
+ cd Lab6/lab6_ejercicios1_2
+ java com.sun.tools.javac.Main -verbose -encoding UTF-8 --release 17 -d
../../qa/build/tienda @fuentes.txt
[wrote ../../qa/build/tienda/module-info.class]
[wrote ../../qa/build/tienda/controlador/TiendaControlador.class]
[wrote ../../qa/build/tienda/modelo/TiendaModelo.class]
[wrote ../../qa/build/tienda/vista/TiendaVista.class]
[wrote ../../qa/build/tienda/modelo/UsuarioModelo.class]
[wrote ../../qa/build/tienda/controlador/UsuarioControlador.class]
[wrote ../../qa/build/tienda/vista/UsuarioVista.class]
[wrote ../../qa/build/tienda/modelo/Usuario.class]
[wrote ../../qa/build/tienda/modelo/Producto.class]
[wrote ../../qa/build/tienda/modelo/ItemCarrito.class]
[wrote ../../qa/build/tienda/modelo/Compra.class]
[wrote ../../qa/build/tienda/modelo/Resena.class]
[wrote ../../qa/build/tienda/principal/Main.class]
[total 387ms]
```

Figura 6. Compilación del carrito con usuarios y reseñas. Salida registrada en una ejecución real con Java 17.

Imágenes de ejecución del ejercicio 1

Catálogo de productos

Se agregan un teclado de S/ 120.00 y un mouse de S/ 50.00. El catálogo muestra sus IDs y precios con dos decimales.

```
Opción: 1
Nombre del producto: Teclado
Precio S/: 120
Producto agregado.

Opción: 1
Nombre del producto: Mouse
Precio S/: 50
Producto agregado.

Opción: 2
1 | Teclado | S/ 120.00
2 | Mouse | S/ 50.00
```

Figura 7. Catálogo de productos. Salida registrada en una ejecución real con Java 17.

Carrito descuento y envío

Se prueba retirar el mouse y volver a añadirlo. El carrito contiene dos teclados y un mouse, suma S/ 290.00, descuenta S/ 29.00 y agrega S/ 10.00 de envío local. El total es S/ 271.00.

```
Opción: 5
ID a eliminar: 2
Producto eliminado del carrito.

Opción: 6
Descuento en porcentaje: 10
1 | Teclado | Cantidad: 2 | S/ 240.00
2 | Mouse | Cantidad: 1 | S/ 50.00
Subtotal: S/ 290.00
Descuento: S/ 29.00
Envío: S/ 0.00
Total: S/ 261.00

Opción: 7
Zona (RECOJO/LOCAL/NACIONAL): LOCAL
1 | Teclado | Cantidad: 2 | S/ 240.00
2 | Mouse | Cantidad: 1 | S/ 50.00
Subtotal: S/ 290.00
Descuento: S/ 29.00
Envío: S/ 10.00
Total: S/ 271.00
```

Figura 8. Carrito descuento y envío. Salida registrada en una ejecución real con Java 17.

Compra confirmada e historial

La confirmación S registra la compra. Consultar el historial devuelve las mismas líneas y los mismos importes guardados, incluso después de vaciarse el carrito.

```
Opción: 8
1 | Teclado | Cantidad: 2 | S/ 240.00
2 | Mouse | Cantidad: 1 | S/ 50.00
Subtotal: S/ 290.00
Descuento: S/ 29.00
Envío: S/ 10.00
Total: S/ 271.00
Confirmar compra (S/N): S
Compra realizada.
Compra #1 | 08/10/2026 22:27
1 | Teclado | Cantidad: 2 | S/ 240.00
2 | Mouse | Cantidad: 1 | S/ 50.00
Subtotal: S/ 290.00
Descuento: S/ 29.00
Envío: S/ 10.00
Total: S/ 271.00

Opción: 9
Compra #1 | 08/10/2026 22:27
1 | Teclado | Cantidad: 2 | S/ 240.00
2 | Mouse | Cantidad: 1 | S/ 50.00
Subtotal: S/ 290.00
Descuento: S/ 29.00
Envío: S/ 10.00
Total: S/ 271.00
```

Figura 9. Compra confirmada e historial. Salida registrada en una ejecución real con Java 17.

Imágenes de ejecución del ejercicio 2

Registro e inicio de sesión

Se registra la cuenta de prueba, se intenta entrar con una clave incorrecta y luego se accede con la clave válida. El mensaje de sesión iniciada aparece únicamente en el intento correcto.

```
Opción: 10
Nombre: Piero
Correo: piero@example.com
Contraseña: clave123
Usuario registrado.

Opción: 11
Correo: piero@example.com
Contraseña: incorrecta
Correo o contraseña incorrectos.

Opción: 11
Correo: piero@example.com
Contraseña: clave123
Sesión iniciada.
```

Figura 10. Registro e inicio de sesión. Salida registrada en una ejecución real con Java 17.

Reseña calificación y promedio

El comprador califica el teclado con 5 y deja una reseña. La consulta muestra el autor y el promedio 5.00 sobre 5. Un segundo intento de reseña del mismo usuario y producto es rechazado.

```
Opción: 13
ID a reseñar: 1
Calificación (1 a 5): 5
Reseña: Buen teclado, funciona bien.
Reseña registrada.

Opción: 14
ID del producto: 1
Piero | 5/5 | Buen teclado, funciona bien.
Promedio: 5.00/5

Opción: 13
ID a reseñar: 1
Calificación (1 a 5): 4
Reseña: Segunda reseña
Ya reseñaste este producto.
```

Figura 11. Reseña calificación y promedio. Salida registrada en una ejecución real con Java 17.

Control de compras por usuario

Después de iniciar sesión con Ana, el historial no muestra la compra de Piero. Ana tampoco puede calificar ese producto porque su cuenta no ha realizado una compra que lo contenga.

```
Opción: 9
No tienes compras registradas.

Opción: 13
ID a reseñar: 1
Calificación (1 a 5): 5
Reseña: No lo compré
Solo puedes reseñar productos que hayas comprado.
```

Figura 12. Control de compras por usuario. Salida registrada en una ejecución real con Java 17.

Verificación de resultados

Los dos proyectos se compilaron con Java 17. Verificacion ejecutó 53 comprobaciones sobre registros, búsquedas, IDs, estados, historial, autenticación, cantidades, precios, descuentos, envío y reseñas. También se ejecutaron sesiones interactivas que probaron actualización, entradas inválidas,

cancelación de compra, compra confirmada y cambio de usuario. Los resultados de consola se encuentran en pruebas/resultados.txt.

Estas pruebas comprueban el comportamiento implementado y los casos de entrada ensayados. El programa no incorpora persistencia, disponibilidad multiusuario simultánea ni un procesador de pagos; su alcance corresponde a una aplicación local de laboratorio.

Archivos complementarios de la entrega

Archivo: .gitignore

```
**/bin/
**/*.class
**/.metadata/
```

Explicación del archivo: .gitignore

Evita subir bin, los archivos .class y los metadatos de ejecución de Eclipse. Estos archivos se regeneran al compilar y no forman parte del código fuente. La entrega conserva los archivos de configuración que permiten importar los proyectos y excluye los resultados temporales del IDE.

Archivo: pruebas/Verificacion.java

```
import java.math.BigDecimal;
import modelo.EstadoPedido;
import modelo.PedidoModelo;
import modelo.TiendaModelo;
import modelo.UsuarioModelo;
import modelo.Compra;
import modelo.Usuario;

public class Verificacion {
  private static int cantidad;

  private static void comprobar(boolean resultado, String nombre)
  {
      if(!resultado)
      {
          throw new AssertionError(nombre);
      }
      cantidad++;
      System.out.println("OK " + cantidad + " - " + nombre);
  }

  private static void rechazar(Runnable accion, String nombre)
  {
      try
      {
          accion.run();
      }
      catch(IllegalArgumentException | ArithmeticException e)
      {
          comprobar(true, nombre);
```

```
            return;
        }
        throw new AssertionError(nombre);
    }

    public static void main(String[] args)
    {
        PedidoModelo pedidos = new PedidoModelo();
        comprobar(pedidos.getPedidos().isEmpty(), "Lista inicial vacía");
        rechazar(() -> pedidos.agregarPedido(" ", "Fondo"), "Nombre vacío rechazado");
        rechazar(() -> pedidos.agregarPedido("Sopa", " "), "Tipo vacío rechazado");
        pedidos.agregarPedido("Sopa", "Entrada");
        pedidos.agregarPedido("Pollo", "Fondo");
        comprobar(pedidos.buscarPedido("sopa").size() == 1, "Búsqueda por nombre");
        comprobar(pedidos.buscarPedidoTipo("fondo").size() == 1, "Búsqueda por tipo");
        comprobar(pedidos.buscarPedido("Otro").isEmpty(), "Búsqueda sin resultados");
        pedidos.actualizarPedido(2, "Pollo al horno");
        comprobar(pedidos.buscarPorId(2).getNombrePlato().equals("Pollo al horno"),
"Actualización");
        rechazar(() -> pedidos.eliminarPedido(99), "ID inválido rechazado");
        comprobar(pedidos.getPedidos().size() == 2, "El error conserva los pedidos");
        pedidos.completarPedido(1);
        comprobar(pedidos.contarPendientes() == 1, "Contador de pendientes");
        rechazar(() -> pedidos.completarPedido(1), "No se completa dos veces");
        rechazar(() -> pedidos.actualizarPedido(1, "Otra sopa"), "No se modifica un pedido
servido");
        pedidos.eliminarPedido(1);
        comprobar(pedidos.getPedidos().size() == 1, "Eliminación lógica");
        comprobar(pedidos.getHistorial().size() == 2, "Historial conserva ambas transiciones");
        comprobar(pedidos.getHistorial().get(0).getEstado() == EstadoPedido.COMPLETO, "Copia
del estado completo");
        comprobar(pedidos.getHistorial().get(1).getEstado() == EstadoPedido.ELIMINADO,
"Copia del estado eliminado");
        comprobar(pedidos.contarPedidos().get("FONDO") == 1, "Conteo por tipo");
        pedidos.agregarPedido("Jugo", "Bebida");
        comprobar(pedidos.buscarPorId(3).getNombrePlato().equals("Jugo"), "ID estable sin
reutilización");

        UsuarioModelo usuarios = new UsuarioModelo();
        rechazar(() -> usuarios.exigirSesion(), "Acceso sin sesión rechazado");
        rechazar(() -> usuarios.registrar("Piero", "incorrecto", "clave123"), "Correo inválido");
        rechazar(() -> usuarios.registrar("Piero", "piero@example.com", "123"), "Contraseña
corta");
        usuarios.registrar("Piero", "piero@example.com", "clave123");
        rechazar(() -> usuarios.registrar("Otro", "PIERO@example.com", "clave123"), "Correo
duplicado");
        rechazar(() -> usuarios.iniciarSesion("piero@example.com", "otra1234"), "Clave
incorrecta");
        comprobar(usuarios.getUsuarioActual() == null, "Fallo de acceso no abre sesión");
        usuarios.iniciarSesion("PIERO@example.com", "clave123");
        Usuario primero = usuarios.exigirSesion();
```

```
        comprobar(primero.getNombre().equals("Piero"), "Inicio de sesión correcto");

        TiendaModelo tienda = new TiendaModelo();
        rechazar(() -> tienda.agregarProducto("Teclado", new BigDecimal("-1")), "Precio
negativo");
        rechazar(() -> tienda.agregarProducto("Teclado", new BigDecimal("0.001")), "Precio con
más de dos decimales");
        tienda.agregarProducto("Teclado", new BigDecimal("120"));
        tienda.agregarProducto("Mouse", new BigDecimal("50"));
        comprobar(tienda.getProductos().size() == 2, "Catálogo de productos");
        rechazar(() -> tienda.agregarAlCarrito(99, 1), "Producto inexistente");
        rechazar(() -> tienda.agregarAlCarrito(1, 0), "Cantidad cero");
        tienda.agregarAlCarrito(1, 1);
        tienda.agregarAlCarrito(1, 1);
        tienda.agregarAlCarrito(2, 1);
        comprobar(tienda.getCarrito().size() == 2 && tienda.getCarrito().get(0).getCantidad() == 2,
                "Acumulación de unidades");
        tienda.eliminarDelCarrito(2);
        comprobar(tienda.calcularSubtotal().compareTo(new BigDecimal("240")) == 0,
"Eliminación del carrito");
        tienda.agregarAlCarrito(2, 1);
        rechazar(() -> tienda.aplicarDescuento(101), "Descuento inválido");
        tienda.aplicarDescuento(10);
        tienda.seleccionarEnvio("local");
        comprobar(tienda.calcularDescuento().compareTo(new BigDecimal("29")) == 0,
"Descuento exacto");
        comprobar(tienda.calcularTotal().compareTo(new BigDecimal("271")) == 0, "Total con
envío local");
        tienda.seleccionarEnvio("nacional");
        comprobar(tienda.calcularTotal().compareTo(new BigDecimal("286")) == 0, "Envío
nacional");
        rechazar(() -> tienda.seleccionarEnvio("otro"), "Zona inválida");
        rechazar(() -> tienda.realizarCompra(null), "Compra sin sesión");
        rechazar(() -> tienda.agregarResena(primero, 1, 5, "Bien"), "Reseña antes de comprar");
        tienda.seleccionarEnvio("local");
        Compra compra = tienda.realizarCompra(primero);
        comprobar(compra.getTotal().compareTo(new BigDecimal("271")) == 0, "Compra
registrada");
        comprobar(tienda.getCarrito().isEmpty(), "Carrito vaciado después de comprar");
        rechazar(() -> tienda.realizarCompra(primero), "Compra vacía rechazada");
        tienda.agregarAlCarrito(1, 1);
        comprobar(compra.getProductos().get(0).getCantidad() == 2, "Historial independiente del
carrito");
        comprobar(tienda.calcularDescuento().compareTo(BigDecimal.ZERO) == 0, "Descuento
reiniciado");
        rechazar(() -> tienda.agregarResena(primero, 1, 6, "Bien"), "Calificación inválida");
        rechazar(() -> tienda.agregarResena(primero, 1, 5, " "), "Reseña vacía");
        tienda.agregarResena(primero, 1, 5, "Buen teclado");
        comprobar(tienda.promedioProducto(1) == 5, "Promedio de calificaciones");
        rechazar(() -> tienda.agregarResena(primero, 1, 4, "Otra"), "Reseña duplicada");
        usuarios.cerrarSesion();
```

```
        tienda.vaciarCarrito();
        comprobar(usuarios.getUsuarioActual() == null, "Cierre de sesión");
        usuarios.registrar("Ana", "ana@example.com", "clave456");
        usuarios.iniciarSesion("ana@example.com", "clave456");
        Usuario segundo = usuarios.exigirSesion();
        comprobar(tienda.historialUsuario(segundo).isEmpty(), "Historial separado por usuario");
        rechazar(() -> tienda.agregarResena(segundo, 1, 5, "Bien"), "Compra ajena no autoriza
  reseña");
        comprobar(tienda.getCarrito().isEmpty(), "Carrito limpio al cambiar de usuario");
        comprobar(tienda.historialUsuario(primero).size() == 1, "Historial del comprador
  conservado");
        System.out.println("Resultado: " + cantidad + " verificaciones correctas.");
    }
  }
```

Explicación del archivo: Verificacion.java

Esta clase comprueba los modelos sin leer el teclado. main prepara pedidos, usuarios y productos y evalúa registros, búsquedas, IDs estables, cambios de estado, historial, descuentos, envío, autenticación y reseñas. comprobar recibe una condición y aumenta el contador únicamente cuando se cumple; rechazar ejecuta una operación mediante Runnable y exige que termine con IllegalArgumentException o ArithmeticException ante una entrada inválida.

Se incluyen casos de error además de los caminos válidos: un correo duplicado, una clave incorrecta, un ID inexistente, una cantidad fuera de rango, una compra sin productos y una reseña sin compra. También se comprueba que las líneas de compras anteriores no cambien al modificar otro carrito y que el historial de un usuario no incluya las compras de otro. Al finalizar se imprime el número total de comprobaciones correctas.

Resultado de la verificación de los modelos

La ejecución de Verificacion termina con 53 verificaciones correctas. La imagen muestra el tramo final de la salida; pruebas/resultados.txt conserva las 53 comprobaciones completas.

```
OK 42 - Compra vacía rechazada
OK 43 - Historial independiente del carrito
OK 44 - Descuento reiniciado
OK 45 - Calificación inválida
OK 46 - Reseña vacía
OK 47 - Promedio de calificaciones
OK 48 - Reseña duplicada
OK 49 - Cierre de sesión
OK 50 - Historial separado por usuario
OK 51 - Compra ajena no autoriza reseña
OK 52 - Carrito limpio al cambiar de usuario
OK 53 - Historial del comprador conservado
Resultado: 53 verificaciones correctas.
```

Figura 13. Resultado de la verificación de los modelos. Salida registrada en una ejecución real con Java 17.

README.md contiene el alcance de la práctica, los comandos de compilación y ejecución, las reglas del carrito y los enlaces a los entregables. pruebas/resultados.txt conserva la salida completa de Verificacion. En pruebas/evidencias, compilacion_actividad.txt y compilacion_ejercicios.txt registran los comandos y los mensajes del compilador; actividad.txt y ejercicios.txt conservan las sesiones completas de consola. docs/capturas contiene las trece imágenes de esos resultados. Los documentos Word y PDF reúnen el código, su explicación, el cuestionario y la bibliografía.

3. CUESTIONARIO

Pregunta 1

¿Qué es un patrón arquitectónico y cuál es su propósito en el desarrollo de software?

Es una solución reutilizable que organiza los componentes y sus relaciones para un tipo de problema de software. Sirve para orientar decisiones de estructura, comunicación y distribución de responsabilidades. Perry y Wolf (1992) explican la arquitectura mediante elementos, forma y justificación; Gamma et al. (1993) muestran cómo los patrones capturan experiencia de diseño. Un patrón ofrece una base adaptable, mientras que el proyecto debe justificar cómo la aplica.

Pregunta 2

¿Cuáles son algunos ejemplos de patrones arquitectónicos populares?

Entre los ejemplos se encuentran capas, cliente servidor, tuberías y filtros, MVC y microservicios. Las capas separan responsabilidades por nivel; cliente servidor distribuye solicitudes y servicios; tuberías y filtros encadena transformaciones; MVC separa dominio, presentación e interacción. Los microservicios organizan capacidades en servicios con autonomía de despliegue. Los estilos estructurales se describen en Perry y Wolf (1992), MVC en Syromiatnikov y Weyns (2014), y la evolución y dificultades de microservicios en Jamshidi et al. (2018).

Pregunta 3

¿Cómo se diferencia un patrón arquitectónico de una arquitectura de software?

El patrón describe una organización reutilizable; la arquitectura corresponde a las decisiones concretas de un sistema. Indicar que una aplicación usa MVC no define todas sus clases, conectores, persistencia ni despliegue. La arquitectura debe precisar esos elementos y justificar sus restricciones. Por ejemplo, en esta práctica MVC se concreta en modelos en memoria, vistas de consola y controladores con menús. La distinción entre una arquitectura específica y sus estilos se fundamenta en Perry y Wolf (1992).

Pregunta 4

¿Qué consideraciones se deben tener en cuenta al seleccionar un patrón arquitectónico para un proyecto?

Se deben considerar los casos de uso, la complejidad de la interfaz, las cualidades requeridas, la tecnología disponible y el costo de mantener la solución. Syromiatnikov y Weyns (2014) relacionan la elección de los patrones de interfaz con requisitos de calidad y tecnología. También se evalúan despliegue, independencia de equipos y comunicación: Jamshidi et al. (2018) muestran que la autonomía de microservicios incorpora dificultades distribuidas. Para esta práctica, una aplicación local pequeña justifica un MVC de consola.

Pregunta 5

¿Cómo interactúan entre sí el Modelo, la Vista y el Controlador en el patrón MVC?

El usuario realiza una acción; el controlador interpreta esa entrada y solicita una operación al modelo. El modelo aplica las reglas y cambia o consulta su estado. La vista muestra el resultado. La separación del modelo, la presentación y el control de la interacción es descrita por Syromiatnikov y Weyns (2014). Una variante puede comunicar cambios a la vista mediante notificaciones. En este laboratorio el controlador obtiene el resultado y lo pasa a la vista de forma explícita. El modelo no necesita conocer la consola.

Pregunta 6

¿Cuáles son las ventajas de utilizar el patrón MVC en aplicaciones web?

Separar la presentación de la lógica del dominio facilita modificar una interfaz, reutilizar operaciones y probar responsabilidades por separado. Leff y Rayfield (2001) estudian MVC en aplicaciones web y proponen mantener la lógica independiente de decisiones de partición entre cliente y servidor. Singh et al. (2018) analizan Spring MVC y Struts 2 para facilitar el desarrollo de aplicaciones Java. Estas ventajas dependen de mantener la separación: colocar toda la lógica en un controlador reduce su utilidad.

Pregunta 7

¿En qué situaciones podría no ser recomendable usar el patrón MVC?

Puede resultar innecesario para un programa pequeño sin interacción de usuario, como una transformación de archivos ejecutada por lotes. En interfaces con enlaces intensivos entre datos y componentes, otra variante puede expresar mejor la presentación. Syromiatnikov y Weyns (2014) indican que la elección debe atender casos de uso y tecnología, y Fuksa et al. (2025) documentan variantes y compromisos de MVVM. Aplicado a esta pregunta, la conclusión es que dividir artificialmente cualquier programa en tres partes no garantiza un mejor diseño.

Pregunta 8

¿Cómo se puede implementar el patrón MVC en aplicaciones de escritorio?

Se crean clases del dominio independientes de la interfaz, una vista que presenta componentes y un controlador que conecta las acciones con operaciones del modelo. La separación de responsabilidades se fundamenta en Syromiatnikov y Weyns (2014). Como aplicación de ese principio a Java, con Swing los botones y tablas pueden pertenecer a la vista y sus acciones llamar al controlador; con JavaFX se puede separar la vista FXML de las clases del dominio. Este laboratorio utiliza consola para practicar el mismo flujo sin añadir una interfaz gráfica.

Pregunta 9

¿Qué otros patrones similares a MVC existen y en qué se diferencian en el contexto de Java?

MVP utiliza un presentador para coordinar la presentación y las acciones de una vista, lo que permite sustituirla en pruebas. MVVM incorpora un ViewModel que expone datos y operaciones de presentación y suele utilizar enlaces de datos. Syromiatnikov y Weyns (2014) comparan estas familias; Fuksa et al. (2025) precisan que MVVM tiene distintas variantes y compromisos. En Java, una interfaz Swing puede organizarse con un presentador y una interfaz JavaFX puede aprovechar propiedades y bindings. Son decisiones de implementación, no restricciones del lenguaje.

Pregunta 10

¿Cómo se pueden combinar patrones de diseño con arquitectónicos en un proyecto Java?

El patrón arquitectónico organiza el sistema y los patrones de diseño resuelven colaboraciones locales dentro de esa organización. Gamma et al. (1993) describen los patrones como estructuras reutilizables que contribuyen a una arquitectura mayor. Por ejemplo, dentro de MVC, Observer puede notificar cambios del modelo, Strategy puede intercambiar reglas de descuento y Factory Method puede organizar la creación de objetos. Se incorporan cuando existe una necesidad concreta; en este carrito se mantuvo un cálculo directo porque la guía solo exige un descuento porcentual.

Pregunta 11

¿Qué herramientas y frameworks Java facilitan la implementación de patrones de diseño y arquitectónicos?

Spring MVC y Struts 2 ofrecen infraestructura para organizar el manejo de solicitudes y la presentación en aplicaciones web Java; Singh et al. (2018) analizan ambos al formular un framework MVC. Como aplicación práctica a escritorio, Swing y JavaFX permiten construir vistas y gestionar acciones. Eclipse facilita organizar paquetes, compilar, depurar y ejecutar proyectos. Un IDE apoya el trabajo, pero la separación arquitectónica depende de las responsabilidades del código. Los proyectos se configuraron para Eclipse y se verificaron desde la terminal con la biblioteca estándar, sin dependencias externas.

4. BIBLIOGRAFÍA

Fuksa, M., Speth, S., & Becker, S. (2025). MVVM revisited: Exploring design variants of the Model-View-ViewModel pattern. En J. Borbinha, T. P. Sales, M. M. da Silva, H. A. Proper, & M. Schnellmann (Eds.), *Enterprise design, operations, and computing* (pp. 163-181). Springer. https://doi.org/10.1007/978-3-031-78338-8_9

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1993). Design patterns: Abstraction and reuse of object-oriented design. En O. M. Nierstrasz (Ed.), *ECOOP 93 Object-oriented programming* (pp. 406-431). Springer. https://doi.org/10.1007/3-540-47910-4_21

Jamshidi, P., Pahl, C., Mendonça, N. C., Lewis, J., & Tilkov, S. (2018). Microservices: The journey so far and challenges ahead. *IEEE Software, 35*(3), 24-35. https://doi.org/10.1109/MS.2018.2141039

Leff, A., & Rayfield, J. T. (2001). Web-application development using the Model/View/Controller design pattern. *Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference*, 118-127. https://doi.org/10.1109/EDOC.2001.950428

Perry, D. E., & Wolf, A. L. (1992). Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes, 17*(4), 40-52. https://doi.org/10.1145/141874.141884

Singh, A., Chawla, P., Singh, K., & Singh, A. K. (2018). Formulating an MVC framework for web development in JAVA. *2018 2nd International Conference on Trends in Electronics and Informatics ICOEI*, 926-929. https://doi.org/10.1109/ICOEI.2018.8553746

Syromiatnikov, A., & Weyns, D. (2014). A journey through the land of Model-View-design patterns. *2014 IEEE/IFIP Conference on Software Architecture*, 21-30. https://doi.org/10.1109/WICSA.2014.13

ANEXO

Repositorio de GitHub

Repositorio del laboratorio: https://github.com/Crudo27/Lab6

El repositorio contiene los proyectos lab6_actividad1 y lab6_ejercicios1_2, sus archivos de configuración para Eclipse, las pruebas, las evidencias y este informe en Word y PDF. Cada proyecto tiene su propia clase principal.Main y se ejecuta por separado.

Compilación y ejecución

Desde la carpeta de cada proyecto, con JDK 17 o superior, se utilizan los siguientes comandos:

```
javac -encoding UTF-8 --release 17 -d bin @fuentes.txt
java -cp bin principal.Main
```

En la verificación se invocó java com.sun.tools.javac.Main porque el entorno dispone del módulo jdk.compiler, pero no del lanzador javac. Ambos comandos utilizan el compilador de Java. En Eclipse se importa cada carpeta mediante File > Import > Existing Projects into Workspace y se ejecuta principal.Main como Java Application.